CS356: DPU-based Accelerator for Deep Neural Network

Hao Liu 185896
hao.liu@kaust.edu.sa
King Abdullah University of Science and Technology
Thuwal, Saudi Arabia

1 INTRODUCTION

AI-based edge computing has been widely used in autonomous driving [13] and underwater robot [20]. Compared with CPUs and GPUs, field-programmable gate array (FPGA) and Data Processing Unit (DPU) which has higher energy efficiency are more suitable for the energy-limited environment. A comparison between CPU, GPU, FPGA and DPU can be seen in table 1. However, deep neural network models (DNN) contain hundreds of millions of parameters and the size of the DNN models can be hundreds of Megabytes. Besides, the floating point operations (FLOPs) can reach billions. For example, AlexNet has 244MB parameter and requires 1.4 GFLOPs per image[12]. The computation and memory requirement make it hard to deploy DNN on FPGAs and DPUs. Quantization is regarded as one of the most efficient ways to solve this problems. By using low-bit points instead of 32-bit floating point weight, the size of DNNs can be compressed and the computation overhead can be reduced[8]. Many methods have been proposed to quantize the DNN models, including posting-training quantization like binary neural networks [2] and quantization aware training[9]. Generally, quantization-aware training can always achieve model accuracy than posting-training quantization[18]. In this project, we plan to use quantization-aware training to quantize the DNN models and then deploy the low-bitwidth model on DPU.

Table 1: Hardware Comparison for DNN

hardware	advantages	disadvantages
GPU	High performance, general-purpose	Higher power consumption, expensive
CPU	General-purpose, reletively cheap	Lower performance, limited parallel processing capabilities
DPU	High performance, lower power consumption, lower latency	Potentially higher cost, limited availability
FPGA	High performance, low power consumption, lower latency	Potentially higher cost, higher development complexity

Many tools have been proposed to deploy DNNs on FPGAs, including Vitis-AI[22], FINN[19] and hls4ml[7]. Vitis-AI and FINN are both developed by Xilinx for the deployment of DNN on FPGA. FINN and hls4ml both target to conduct the inference of low precision DNN by building dataflow inference engines into FPGA fabric. Different from FINN and hls4ml, Vitis-AI is proposed to implement a time-multiplexed layer-by-layer DNN inference by using the DPU overlay on FPGA. Many libraries have been proposed for quantization-aware training by xilinx for further implementing on FPGA, including Brevitas[17], QKeras quantization-aware training of hls4ml and vai_q library of Vitis-AI. Compared with Brevitas and Qkeras which can implement more options of bitwidth neural networks, Vitis-AI supports a limited options of bitwidth, which can only be 4, 8 or 16. The comparison of Vitis-AI, FINN and hls4ml can be seen in table 2.

Table 2: Comparison of existing tools

	Vitis-AI	FINN	hls4ml
Target	DPU	FPGA fabric	FPGA fabric
Supported ML framework	Tensorflow, Pytorch, Caffe	Pytorch	Tensorflow, Keras, Pytorch
Quantization library	vai_q_pytorch, vai_q_tensorflow	Brevitas	Qkeras
Quantization aware training	yes	yes	yes
Supported bitwidth	4, 8, 16	flexible	flexible

Image classification is wildly used in autonomous driving and medical diagnosis. Cifar10 and cifar100[11] are two common classification dataset. Cifar10 contains 60000 images in 10 classes and cifar100 dateset contains 100 categories of objects containing 600 images each. ResNet has been proved to be an efficient model for classification tasks. In this project, we plan to deploy cifar100-based quantized ResNet-18 and ResNet-50 model on DPU by Vitis-AI. we will explore the performance and resource requirement of quantized model with different bitwidth. Besides, we will deploy cifar10-based quantized ResNet-20 on DPU and Alevo U280 to compare the performance of DPU and FPGA. The whole project is open-source and the link is https://github.com/liuhao-97/cs356_project/tree/main.

In summary, our contribution includes:

- Quantization aware training: We will pretrain ResNet-18 and ResNet-50 on cifar100 dataset, and ResNet-20 on cifar10 dataset. The full precision pretrained models will be quantized by quantized aware training with different bitwidth.
- Deployment of QNN on DPU: The quantization models will be compiled and deployed on DPU by Vitis-AI. The performance and required resources of different bitwidth models will be compared and analyzed.
- Performance comparison with FPGA: We will further pretrain the ResNet-20 on cifar10 dataset by Keras. The pretrained model will be quantized by hls4ml and further deployed on Alevo u280. The reuse factor will be changed to explored the latency and required resources of the models with different levels of parallel. The performance of FPGA model will be compared with DPU.

2 RELATED WORK

2.1 Quantization neural network on FPGA

Quantization neural network (QNN) is proposed to use low-bit weights and activations to represent full precision parameters. Courbariaux et al.[1] proposed low precision CNNs with binary weight. Courbariaux et al.[2] went further to propose binarized CNNs which use binary weights and activations. However, these QNN implementations are all based on GPUs. FPGA fabric can also gain benefit from QNNs. QNNs with less memory requirement are much easier to fit in the memory of FPGAs and reduce the BW requirement as well[6]. The computing overhead can also diminish by replacing floating points multiply and accumulate (MACs) to low-bit operations. Many works have deployed QNNs and explored the performance and resource requirement on FPGAs. Zhan et al.[24] firstly proposed an accelerator for BNNs on FPGA. Umuroglu et al. [19] proposed FINN to map BNNs for classification on FPGAs with the speed of 12.3 million images per second on MNIST dataset and 21906 images per second on CIFAR-10 and SVHN dataset. Ducasse et al. [5] has explored the impact of mixed-precision when deployed neural networks on FPGA using the MINIST dataset. Based on FINN and Brevitas, they explored both the impact of different bits quantization and different parallelism configurations.

2.2 Parallelism on FPGA

Apart from hardware-friendly algorithm QNN, the architecture optimization can also impact the performance of accelerators. Generally, the higher the parallelism is, the less the resources are reused, the higher the bandwidth is and the better the performance is. Moini et al.[14] proposed a new architecture to exploit the inherent parallelism in CNN to reduce the required bandwidth, resources and energy consumption for embedded vision applications. Different reuse factor of different layers in DNN are explored. Convolution layers has been proved to own the highest reuse factor and thus can gain the most beneficial from parallelism. Motamedi et al. [15] proposed an FPGA-based accelerator with all sources of parallelism in DNN. A model was developed to estimate whether the optimal architecture was feasible and the performance on certain FPGA.

2.3 Related to course

This work is close related to [10]. Norman et al.[10] proposed custom ASIC - Tensor Processing Unit(TPU) to accelerate the execution of NN applications. TPU is designed to corporate with Google's Tensorflow framework, while DPU can support more programming frameworks. Our work is also related to roofline model[21]. Roofline model can give a insight where is the bottleneck of the execution and how to improve the performance. In our work, the bottleneck is the bandwidth.

2.4 Vitis-AI

Vitis-AI is an integrated development environment developed by AMD Xilinx. It can leverage the DPU on FPGA board to accelerate DNNs. The workflow of Vitis-AI is like figure 1. The original DNN model need to be rewriten in the way which can be quantized later by Vitis-AI. The rewriten model are pretrained in Pytorch or Tensorflow framework. Then the pretrained model will be quantized by quantization-aware training with different bitwidth. The quantized models will be transferred to deployable models which follows the data format of DPU. Finally the deployable model will be compiled to .xmodel files by Vitis-AI compiler and be deployed on DPU [23].

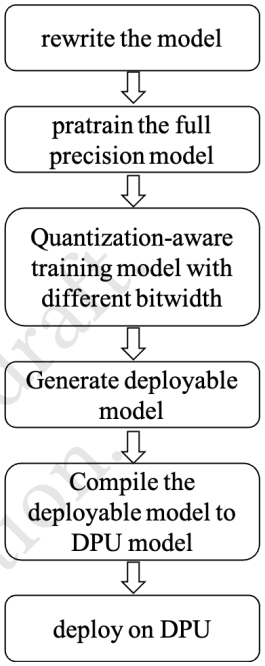


Figure 1: Workflow of Vitis-AI

2.5 Hls4ml

Hls4ml library is a python library which can efficiently transfer the model generated from Pytorch or Keras to high level synthesis (HLS) code[7]. Hls4ml is firstly created for high energy physics community[4]. It is used to generated hardware accelerators to accelerate which data should be kept during one collision. Ngadiuba et al.[16] is the first to propose the implementation of binary and ternary neural networks in the hls4ml library. The workflow of Hls4ml can be seen in figure 2[7]. First the original model trained by Keras, Pytorch or Tensorflow will be compressed by quantization or pruning. Then model will be convert to HLS and users can tune the configuration based on the demand. Finally the FPGA accelerator will be generated.

Hls4ml provides API to configure the reuse factors for FPGA accelerators. The reuse factor can decide the number of times each multiplier is used to conduct computing as figure 3[7]. For example, if the reuse factor is 1, then each multiplier only need to conduct multiplication once and if the reuse factor is 2, then each multiplier need to conduct multiplication twice. Hls4ml also offers a fine-grain size of reuse factor configuration which allows the reuse factors varied layer by layer. By changing the reuse factor, the FPGA accelerator can reach the trade-off between resources and latency. Lower reuse factors means a high level of parallel with low latency but also requires more resources, including LUTs, registers

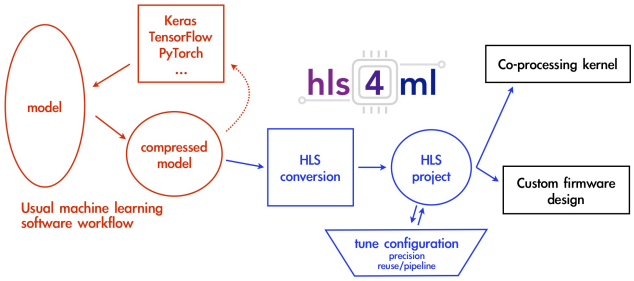


Figure 2: Workflow of Hls4ml

and DSP. By exploring the reuse factor of hls4ml, we can compare the performance of QNN with different configurations.

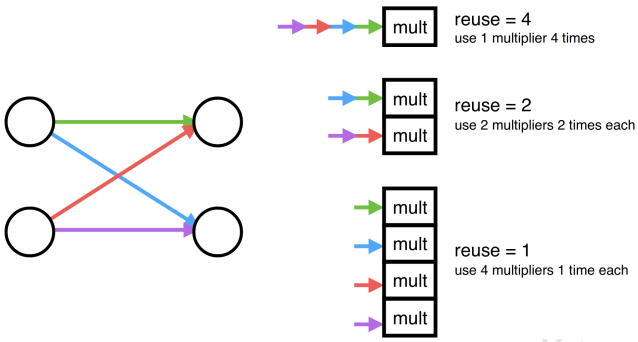


Figure 3: Reuse factor of hls4ml

In our work, We will follow the same workflow with [5] to explore the throughput, bandwidth and resource utilization of QNNs with different quantization on cifar10 and cifar100 dataset by Vitis-AI and hls4ml.

3 EVALUATION

3.1 Experimental setting

For DPU model, the cifar100-based full precision ResNet-18/50, and cifar10-based full precision ResNet-20 are trained by Pytorch1.7. The pretrained models are quantized and compiled by Vitis-AI2.5. The target hardware is Alevo U55C DPU. For FPGA model, the cifar10-based full precision ResNet-20 is trained by Keras. The pretrained model is quantized and compiled by hls4ml0.4.0. The target hardware is Alevo U280.

3.2 Inspect and Predict assignment of DPU

When we use Vitis-AI to deploy the DNNs on DPU, different layers can be offloaded to either DPU or CPU depending on the device architectures[23]. We use inspector to plot the DAG which predicts the target device assignment. The result of ResNet-18 can be seen in figure 4. All the layers can be deployed on DPU except the input layer and the last dequant_stub layer.

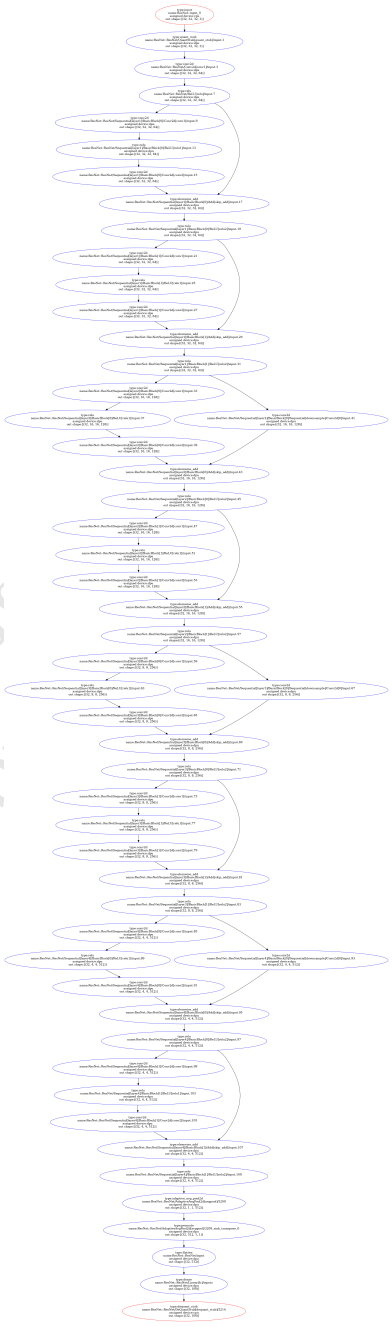


Figure 4: ResNet-18 Inspection

3.3 Quantization Aware Training

As mentioned in figure 1, we pretrain the full precision model and quantize the pretrained model by quantization aware training. The results of ResNet-18 and ResNet-50 on Cifar100 dataset, and ResNet-20 on Cifar10 dataset can be seen in table 3, table 4 and table 5 separately. ResNet-18 can gain a little accuracy increase when using 16-bit or 8-bit quantization. When quantized with 4 bits, ResNet-18 will suffer an accuary decrease of 18.3%. The accuracy

decrease of ResNet-50 is less than 1% when quantized with 16 or 8 bits. When quantized with 4 bits, ResNet-50 will suffer the accuracy drop of 8.48%. For ResNet-20, the accuracy will almost maintain when quantized with 16 bits and 8 bits. When quantized with 4 bits, there will be almost 20% accuracy decrease.

Table 3: Low-bitwidth ResNet-18 Accuracy on Cifar100

Quantization	top1-accuracy	top5-accuracy
full-precision	0.7608	0.9283
16-bit	0.7652	0.9309
8-bit	0.7616	0.9300
4-bit	0.5778	0.8089

Table 4: Low-bitwidth ResNet-50 Accuracy on Cifar100

Quantization	top1-accuracy	top5-accuracy
full-precision	0.7931	0.9442
16-bit	0.7850	0.9446
8-bit	0.7866	0.9442
4-bit	0.7083	0.9044

Table 5: Low-bitwidth ResNet-20 Accuracy on Cifar10

Quantization	top1-accuracy	top5-accuracy
full-precision	0.9239	0.9975
16-bit	0.9260	0.9970
8-bit	0.9235	0.9971
4-bit	0.7257	0.9723

3.4 Execution on DPU

We further generate the deployable models from the quantized models and then compile them to DPU models, which can be deployed on Alevo U55C. The execution result of ResNet-50, ResNet-18 on Cifar100 dataset and ResNet-20 on Cifar10 dataset are shown in table 6, table 7 and table 8. We follow the setting of image number as the demo, e.g. one image for testing. We set the the number of times a single-thread DPU runs to 400 for a better measurement of the average performance. Time in the table is the whole latency of processing the frames of all the threads by DPU. The frames per second (FPS) equals to total_frame devided by time. As we increase the thread number, the throughput of DPU will increase as well. However, the performance of DPU models with different bitwidths are similar. For example, the throughputs of ResNet-50 with 8-bit and 4-bit are 708.67 and 707.74 respectively. Maybe it is because the input image is quite small (3*32*32) and the DPU models are relatively small, the computation required for inference is far away from the computing capacity of DPU. The bottleneck is the bandwidth.

Vitis-AI provides vaitrace tool which can profile the DPU model. Combined with Vitis Analysis tool, we can track the performance of DPU model. The tracking results of ResNet-50 and ResNet-18 can be seen in figure 5 and figure 6 separately.

Table 6: Execution performance of ResNet-50 DPU model

Quantization	thread	FPS	total frames	time (second)
16bit	1	699.54	400	0.571801
	2	1066.69	800	0.74998
	3	1135.45	1200	1.056853
8bit	1	708.67	400	0.564440
	2	1032.14	800	0.775091
	3	1289.75	1200	0.930411
4bit	1	707.74	400	0.56517
	2	1097.53	800	0.728907
	3	1168.40	1200	1.027041

Table 7: Execution performance of ResNet-18 DPU model

Quantization	thread	FPS	total frames	time (second)
16bit	1	1480.49	400	0.270180
	2	1923.45	800	0.415920
	3	2529.10	1200	0.474477
8bit	1	1601.50	400	0.249766
	2	2138.10	800	0.374164
	3	2715.26	1200	0.441946
4bit	1	1570.99	400	0.254617
	2	2236.14	800	0.357759
	3	2655.92	1200	0.451821

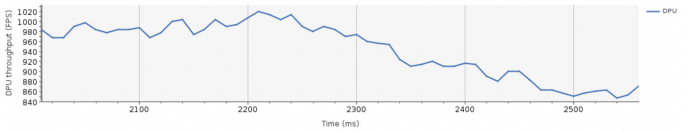
Table 8: Execution performance of ResNet-20 DPU model

Quantization	thread	FPS	total frames	time (second)
16bit	1	4386.13	400	0.091197
	2	6568.12	800	0.121800
	3	7293.36	1200	0.164533
8bit	1	4494.59	400	0.088996
	2	6394.57	800	0.125106
	3	7837.24	1200	0.153115
4bit	1	4518.20	400	0.088531
	2	5830.60	800	0.137207
	3	7949.73	1200	0.150949

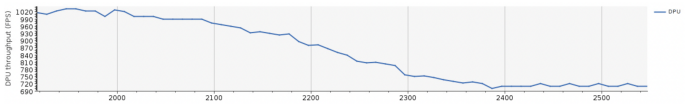
Finally as a comparison, we also run the DPU model of ResNet-50 trained on the ImageNet dataset[3]. The size of input image is 3*224*224. The DPU model reaches 289.79 FPS and the time for 400 frames is 1.38 seconds. The performance tracking result can be seen in figure 7.

3.5 Comparison between DPU and FPGA

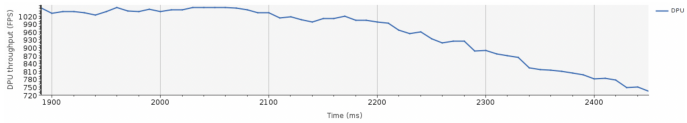
In order to compare the performance of DPU and FPGA, we choose to deploy ResNet-20 on both DPU and FPGA as ResNet-18 and ResNet-50 are too big to deploy on FPGA. For FPGA model, we use hls4ml to generate the bitstream. Our target hardware is Alevo U280. The accuracy of pretrained model and quantized model can be seen in table 9. Ap_fix<16,6> means a total bitwidth of 16 and 6 fractional bits. As a comparision, we can see that the FPGA model suffers more accuracy drop than DPU model.



(a) 16-bit ResNet-50

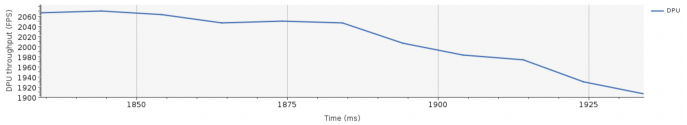


(b) 8-bit ResNet-50

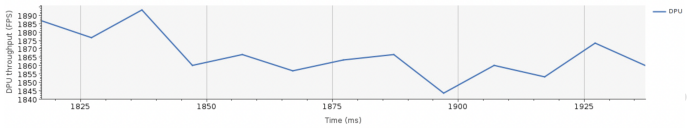


(c) 4-bit ResNet-50

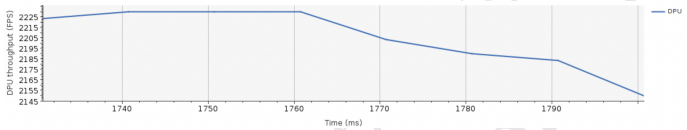
Figure 5: ResNet-50 performance tracking



(a) 16-bit ResNet-18



(b) 8-bit ResNet-18



(c) 4-bit ResNet-18

Figure 6: ResNet-18 performance tracking

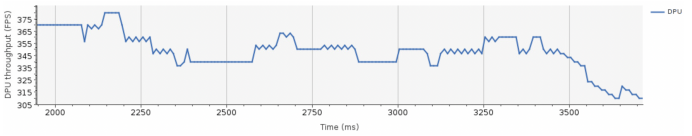


Figure 7: ImageNet-based ResNet-50 performance tracking

Table 9: ResNet-20 Accuracy on Cifar10

tools	Quantization	top-1 accuracy
Vitis-AI	full-precision	0.9239
	16-bit	0.9260
hls4ml	full-precision	0.9184
	ap_fix<16,6>	0.877

We try both high and low reuse factors for ResNet-20. The estimated latency and resource utilization can be seen in table 10 and table 12 seperately. Compared with high reuse factor, the FPGA model with low reuse factor has lower latency and requires more LUTs for logic and DSP resources. The DSP resources required by both designs are not varied a lot. It is because the low reuse factor is half of the high reuse factor. The high and low reuse factors are still close to each other. DPU accelerator is 1119x-1860.5x faster than FPGA accelerator with high reuse factor and 752.21x-1250.8x faster than FPGA accelerator with low reuse factor.

Table 10: Estimate latency of ResNet-20 ap_fix<16,6>

reuse factor	latency (cycles)		latency (absolute)		Interval	
	min	max	min	max	min	max
High	116383515	118079243	0.253 sec	0.257 sec	20482	117193836
Low	39741013	40552004	0.170 sec	0.173 sec	20482	40551401

Table 11: Performance comparison of ResNet-20 between DPU and FPGA

Hardware	thread	reuse factor	throughput (FPS)	latency (ms)
DPU	1	N/A	4386.13	0.228
	2	N/A	6568.12	0.15225
	3	N/A	7293.36	0.13711
FPGA	N/A	High	3.92	255
	N/A	Low	5.831	171.5

Table 12: Resource Utilization of ResNet-20 ap_fix<16,6>

Resouces type	Available	High reuse factor		Low reuse factor	
		Used	Util%	Used	Util%
CLB LUTs*	1303680	531968	40.81	550526	42.23
LUT as Logic	1303680	**375072**	**28.77**	**393628**	**30.19**
LUT as Memory	600960	156896	26.11	156898	26.11
CLB Registers	2607360	550256	21.10	554710	21.27
DSP	9024	**702**	**7.78**	**721**	**7.99**

4 FUTURE WORK

- ImageNet-based DPU model: We will generate more DPU models for ImageNet dataset, e.g. VGG-16 in order to further explore the performance of DPU.
- Experiment with Vitis AI optimizer: Vitis-AI provides pruner tools to optimize the model. It can prune the less significant connections and reduce the computation without large accuracy drop. Besides, Hls4ml also provides tools to prune the model. We will explore the performance of pruned DPU model and compare with FPGA accelerator in the future.

REFERENCES

[1] Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. 2015. Binaryconnect: Training deep neural networks with binary weights during propagations. *Advances in neural information processing systems* 28 (2015).

[2] Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Binarized neural networks: Training deep neural networks with weights and activations constrained to+ 1 or-1. *arXiv preprint arXiv:1602.02830* (2016).

[3] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 248–255.

[4] Javier Duarte et al. 2018. Fast inference of deep neural networks in FPGAs for particle physics. *JINST* 13, 07 (2018), P07027. https://doi.org/10.1088/1748-0221/13/07/P07027 arXiv:1804.06913 [physics.ins-det]

[5] Quentin Ducasse, Pascal Cotret, Loïc Lagadec, and Rob Stewart. 2021. Benchmarking Quantized Neural Networks on FPGAs with FINN. In *DATE Friday Workshop on System-level Design Methods for Deep Learning on Heterogeneous Architectures*.

[6] Mohsine Eleuldj et al. 2020. Survey of deep learning neural networks implementation on FPGAs. In *2020 5th International Conference on Cloud Computing and Artificial Intelligence: Technologies and Applications (CloudTech)*. IEEE, 1–8.

[7] FastML Team. 2021. *fastmachinelearning/hls4ml*. https://doi.org/10.5281/zenodo.1201549

[8] Yunhui Guo. 2018. A survey on methods and theories of quantized neural networks. *arXiv preprint arXiv:1808.04752* (2018).

[9] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2017. Quantized neural networks: Training neural networks with low precision weights and activations. *The Journal of Machine Learning Research* 18, 1 (2017), 6869–6898.

[10] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.

[11] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images. (2009).

[12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2017. Imagenet classification with deep convolutional neural networks. *Commun. ACM* 60, 6 (2017), 84–90.

[13] Shaoshan Liu, Liangkai Liu, Jie Tang, Bo Yu, Yifan Wang, and Weisong Shi. 2019. Edge computing for autonomous driving: Opportunities and challenges. *Proc. IEEE* 107, 8 (2019), 1697–1716.

[14] Shayan Moini, Bijan Alizadeh, Mohammad Emad, and Reza Ebrahimpour. 2017. A resource-limited hardware accelerator for convolutional neural networks in embedded vision applications. *IEEE Transactions on Circuits and Systems II: Express Briefs* 64, 10 (2017), 1217–1221.

[15] Mohammad Motamedi, Philipp Gysel, Venkatesh Akella, and Soheil Ghiasi. 2016. Design space exploration of FPGA-based deep convolutional neural networks. In *2016 21st Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 575–580.

[16] Jennifer Ngadiuba et al. 2021. Compressing deep neural networks on FPGAs to binary and ternary precision with HLS4ML. *Mach. Learn. Sci. Tech.* 2 (2021), 015001. https://doi.org/10.1088/2632-2153/aba042 arXiv:2003.06308 [cs.LG]

[17] Alessandro Pappalardo. 2022. *Xilinx/brevitas*. https://doi.org/10.5281/zenodo.3333552

[18] Tensorflow contributor. 2022. Quantization aware training. https://www.tensorflow.org/model_optimization/guide/quantization/training.

[19] Yaman Umuroglu, Nicholas J Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. 2017. Finn: A framework for fast, scalable binarized neural network inference. In *Proceedings of the 2017 ACM/SIGDA international symposium on field-programmable gate arrays*. 65–74.

[20] Lin Wang, Xiufen Ye, Shunli Wang, and Peng Li. 2022. ULO: An Underwater Light-Weight Object Detector for Edge Computing. *Machines* 10, 8 (2022), 629.

[21] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (2009), 65–76.

[22] AMD Xilinx. 2023. Vitis-AI. https://github.com/Xilinx/Vitis-AI.

[23] AMD Xilinx. 2023. Vitis-AI User Guide. https://docs.xilinx.com/r/2.5-English/ug1414-vitis-ai/Vitis-AI-Quantizer-Flow.

[24] Ritchie Zhao, Weinan Song, Wentao Zhang, Tianwei Xing, Jeng-Hau Lin, Mani Srivastava, Rajesh Gupta, and Zhiru Zhang. 2017. Accelerating binarized convolutional neural networks with software-programmable FPGAs. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 15–24.